Timbr 매핑용 데이터소스 안내 — 부트캠프 3·4번(제조·상품거래)

> **용도**: 9/9 부트캠프(기초) 실습 5사례 중 **3 제조 · 4 상품거래**의 데이터가 Timbr에 어떻게 올라가 있는지, 개념·매핑을 만들 때 무엇을 알아야 하는지 한 곳에 정리한 문서. Timbr 콘솔에서 매핑하는 사람이 **이 문서만 보고** 시작할 수 있게 썼다. **작성**: 박종범, 2026-09-03. 데이터 적재·데이터소스 연결은 9/2~9/3에 끝났다. 아래 수치는 적재된 DB와 적재 원본을 다시 세어 확인한 값이고(부록 A에 조회문), §5의 SQL은 9/3에 임시 온톨로지에서 전부 실행해 행수를 확인했다(부록 B). 1·2번(통신·금융)은 7/16 자료로 이미 구축돼 있고 (`telecom_cdr_kg` / `bank_txn_kg`), 5번(보안)은 기존 SOC 온톨로지를 쓴다 — 이 문서의 범위 밖.

0. 한눈 요약

사례	Timbr 온톨로지(KG)	데이터소스	Postgres 스키마.테이블	행 × 열	기간
3 제조 — 장애 예측	`3_manufacture`	`bootcamp_supabase`	`manufacture.ai4i_predictive_maintenance`	10,000 × 14	시간 컬럼 없음 (공정 사이클 1 만 건)
4 상품거래 — 원가 예측	`4_commodity_trading`	`bootcamp_supabase` (공용)	`commodity_trading.prices_monthly`	799 × 72	1960-01 ~ 2026-07 월별
			`commodity_trading.indices_monthly`	799 × 17	1960-01 ~ 2026-07 월별
			`commodity_trading.prices_legacy`	449 × 64	1980-02 ~ 2017-06 월별(구식 스냅샷,

사례	Timbr 온톨로지(KG)	데이터소스	Postgres 스키마.테이블	행 × 열	기간
					출처 다름 — §2)

- **매핑에 필요한 준비는 끝나 있다**: 두 온톨로지 모두 데이터소스 `bootcamp_supabase`가 연결·활성 상태다. 데이터소스를 새로 만들지 말 것(§8).
- **모든 컬럼은 Postgres에 TEXT로 들어 있다.** 매핑에서 `CAST`가 필요하다(§4). 기존 통신·금융 사례와 같은 방식이다.
- 데이터는 전부 **공개 데이터**이고, 제조 데이터는 **합성(synthetic)** 데이터다(§2). 고객 데이터가 아니다.

1. Timbr에서 보이는 구조

- 워크스페이스: `kosena.timbr.ai`
- 온톨로지(Knowledge Graph) 2개: `3_manufacture`, `4_commodity_trading` — 둘 다 **개념 0개(루트 `thing` 만)**인 빈 그래프. 여기에 개념·매핑을 만든다.
- 데이터소스: `bootcamp_supabase` 1개(PostgreSQL, Supabase). 두 온톨로지에 **같은 데이터소스가 연결**돼 있고, 각 온톨로지에서 `SET active_datasource`를 실행해 `SHOW DATASOURCES`의 `is_active`가 1이다. 이 설정이 영구적인지는 공식 문서에 없으므로(9/3 실측으로 REST 호출 사이에 유지되는 것만 확인) 콘솔 `Manage → Knowledge Graphs → Datasources`의 **Default active datasource**에서 한 번 더 확인하면 좋다.
- 이 데이터소스에는 위 4개 테이블 외에 통신·금융·SOC 사례의 스키마(`telecom_cdr`, `bank_txn`, `ocsf_aws` 등)도 같은 DB에 있어 **스키마 목록에 함께 보인다**. 제조는 `manufacture`, 상품거래는 `commodity_trading` 스키마만 쓰면 된다.

원본 테이블 확인(SQL Lab): **상단 SCHEMA 드롭다운을 `manufacture` 또는 `commodity_trading`으로 맞추고** 실행한다 — 드롭다운이 쿼리 안의 스키마 표기보다 우선한다(기존 사례에서 실측). 아래 값은 9/2~9/3 REST로 확인한 값이며 SQL Lab UI에서는 아직 실행하지 않았다.

```
SELECT COUNT(*) FROM "manufacture"."ai4i_predictive_maintenance";      -- 10000
SELECT COUNT(*) FROM "commodity_trading"."prices_monthly";            -- 799
```

2. 데이터 성격·출처·이용 조건

데이터	출처	성격	이용 조건·표기
AI4I 2020 Predictive Maintenance	UCI Machine Learning Repository, dataset 601 — https://archive.ics.uci.edu/dataset/601 (DOI 10.24432/C5HS5C). 적재 파일은 공식 zip과 바이트 동일(SHA-256 대조)	**합성 데이터** — UCI 원문 "a synthetic dataset that reflects real predictive maintenance data encountered in	CC BY 4.0. 표기: *AI4I 2020 Predictive Maintenance Dataset [Dataset]. (2020). UCI Machine Learning Repository. https://doi.org/10.24432/C5HS5C* (소개 논문: S. Matzka 2020, DOI 10.1109/AI4I49448.2020.00023 — 저자는 논문 저자이지 데이터셋 제작자로 등록돼 있지 않다)

데이터	출처	성격	이용 조건·표기
		industry". 실제 설비 센서값이 아니다. 발표에서 "공개 합성 예지보전 데이터"라고 밝힐 것	
World Bank Commodity Price Data (The Pink Sheet)	worldbank.org/en/research/commodity-markets → `CMO-Historical-Data-Monthly.xlsx`, **2026-08-04 갱신본**(공식 배포본과 SHA-256 동일)	실제 국제 원자재 가격·지수. 명목 USD, 월별. World Bank는 매월 새 파일을 올리고 **파일 URL의 문서 ID가 바뀐다**(구 URL은 낡은 파일을 그대로 준다) — 재적재 시 페이지의 현재 링크를 다시 딴다. 우리 적재본은 "2026-08-04 갱신본 (2026-07까지)" 스냅샷	World Bank 자체 데이터셋 기본 라이선스는 CC BY 4.0(Data Catalog 표기). **단, 개별 시계열 일부는 Bloomberg·S&P Global 등 제3자 출처**(xlsx Description 시트 Sources란)라 원자료 **재배포 조건은 미확인** — 발표 슬라이드의 집계·차트 인용은 무방하나 파일 자체를 외부에 재배포하지 않는다. 표기 형식(World Bank 규정): *The World Bank: Commodity Price Data (The Pink Sheet), August 4, 2026 update: World Bank Prospects Group*
`commodity-prices.csv` (레거시)	GitHub `datasets/commodity-prices` — README "Data comes from the International Monetary Fund (IMF)"(원 소스는 IMF 외부데이터 xls). **IMF 계열이며 World Bank 자료가 아니다**	지수 정의·기준연도(2005=100, 실측으로 확인)·상품 구성이 Pink Sheet와 다르다 → **두 계열을 한 시계열로 이어 붙이지 말 것**. 같은 리포지토리에 v2(1990-01~2025-09, 2016=100)도 있으나 적재하지 않았다	**라이선스 상충**: README는 "All rights are reserved" + IMF 약관(개인·비상업 한정, 재배포·파생물 불가), `datapackage.json`은 ODC-PDDL. 보수적으로 IMF 약관을 따라 **부트캠프 내부 실습 한정 — 외부 발표 자료·재배포에 쓰지 않는다**

3. 테이블·컬럼 사전

3-1. `manufacture.ai4i_predictive_maintenance` — 10,000행 × 14열

한 행 = 공정 사이클 1건. 시간 컬럼은 없다(UCI 태그는 Time-Series이지만 파일에 시각은 없다). 원본 컬럼명을 그대로 뒀기 때문에 **공백·대괄호가 들어 있다 → Timbr SQL에서는 백틱으로 감싼다**(`` `Air temperature [K]` ``). UCI 설명문은 첫 열을 `UID`라 쓰지만 파일 헤더는 `UDI`다.

컬럼(원본명 그대로)	뜻	값·단위	권장 CAST	실측
`UDI`	행 고유번호	1~10000	int — **Primary Key 권장**	고유 10,000
`Product ID`	제품 일련번호	품질등급 문자(L/M/H) + 숫자, 예 `M14860`. 첫 글자 = `Type`	string — Entity Label 후보	고유 10,000(행마다 다름)
`Type`	제품 품질 등급	`L` (low) / `M` (medium) / `H` (high)	string	L 6,000 · M 2,997 · H 1,003 (60/30/10%). **UCI 설명문의 "50/30/20%"는 실제 파일과 다르다 — 원문 비율을 인용하지 말 것**
`Air temperature [K]`	주변 공기 온도	켈빈(K)	double	295.3 ~ 304.5
`Process temperature [K]`	공정 온도	켈빈(K)	double	305.7 ~ 313.8
`Rotational speed [rpm]`	회전 속도	rpm	int	1,168 ~ 2,886
`Torque [Nm]`	토크	N·m	double	3.8 ~ 76.6
`Tool wear [min]`	공구 누적 마모 시간	분	int	0 ~ 253
`Machine failure`	고장 라벨	`0` / `1`	int 또는 boolean	339건(339/10,000 = 3.4%). **아래 5유형과 정확히 대응하지 않는다(다음 단락)**
`TWF`	공구 마모 고장 (tool wear failure)	`0` / `1`	int/boolean	46 (전건 라벨 1)
`HDF`	방열 고장(heat dissipation failure)	`0` / `1`	int/boolean	115 (전건 라벨 1)
`PWF`	전력 고장 (power failure)	`0` / `1`	int/boolean	95 (전건 라벨 1)
`OSF`	과부하 고장 (overstrain failure)	`0` / `1`	int/boolean	98 (전건 라벨 1)
`RNF`	무작위 고장 (random failure)	`0` / `1`	int/boolean	19 — **그중 18건은 `Machine failure` =0**

라벨과 유형 플래그의 실제 관계(실측) — UCI 설명은 "5유형 중 하나라도 참이면 `Machine failure`=1"이지만 파일은 그렇지 않다. 유형 플래그가 하나라도 1인 행 = 348, `Machine failure`=1인 행 = 339. 차이는 (a) RNF=1이면서 라벨 0인 행 18건, (b) 라벨 1이면서 플래그가 하나도 없는 행 9건. 유형 플래그 합 373 = 유형 있는 행 348 + 중복 표시 25(2유형 23행·3유형 1행). 매핑 함의: `failed_cycle`(라벨=1, 339행)과 그 아래 HDF/PWF/OSF/TWF 하위개념은 성립하지만, **RNF 하위개념을 `failed_cycle` 아래에 두면 1행만 나온다** — RNF는 `process_cycle`에서 직접 파생할 것.

고장 생성 규칙(UCI 원문, 실습 소재) — 생성기 설명이며, 아래 세 규칙은 파일에서 **정확히 재현된다**(Postgres에서 계산해 플래그와 1:1 대조):

유형	UCI 원문 규칙	재현 여부(실측)
HDF	공정온도 − 공기온도 < 8.6 K **and** 회전속도 < 1380 rpm	규칙 충족 115 = HDF 플래그 115, 전부 일치
PWF	전력 = 토크 × 회전속도(rad/s); 3500 W 미만 또는 9000 W 초과	규칙 충족 95 = PWF 95, 전부 일치
OSF	공구마모 × 토크 > 11,000 minNm(L) / 12,000(M) / 13,000(H)	규칙 충족 98 = OSF 98, 전부 일치
TWF	공구마모 200~240분 구간에서 무작위로 교체 또는 고장(원문 "fails 51 times")	플래그 46건, 마모 범위 198~253 — 원문 수치와 다름
RNF	각 사이클 0.1% 확률(원문 "only 5 datapoints")	플래그 19건 — 원문 수치와 다름

생성 방식(원문): 공기온도는 300 K 주변 σ 2 K 랜덤워크, 공정온도 = 공기온도 + 10 K(σ 1 K), 회전속도는 2860 W 기준에 정규 노이즈, 토크는 평균 40 Nm σ 10 Nm(음수 없음), 공구마모는 사이클마다 H/M/L 등급별 5/3/2분 누적. → HDF/PWF/OSF는 **속성 계산식만으로 하위개념을 만들어 플래그와 같은 행수가 나오는지** 보여 주기 좋다(§5-B 끝의 선택 블록).

- 실습 시나리오 문서가 제안한 분해(`machine, product, sensor_observation, failure_event, maintenance_event`)는 이 테이블에 설비 ID·정비 이력이 없고 `Product ID`가 행마다 고유해 product↔observation이 1:1이 된다. 관계를 보여 주려면 §5-B처럼 `Type`(등급)을 참조 개념으로 떼는 편이 데이터에 맞는다.

3-2. `commodity_trading.prices_monthly` — 799행 × 72열 (World Bank, 명목 USD)

한 행 = 한 달. `period_date`는 `'YYYY-MM-01'` 문자열. 71개 상품 컬럼은 xlsx "Monthly Prices" 시트의 열 이름을 스네이크케이스로 바꾼 것이다. **단위는 상품마다 다르다**(아래, xlsx 단위행 그대로). 결측은 NULL — 원본의 `…`(값 없음) 6,184셀, `...`(DAP 1960~66) 84셀, `#VALUE!`(엑셀 오류, `rice_thai_25` 4셀)를 구분 없이 NULL로 넣었다.

분류(Pink Sheet 지수 그룹)	컬럼(단위)
에너지	`crude_oil_average` `crude_oil_brent` `crude_oil_dubai` `crude_oil_wti` ($/bbl) · `coal_australian` `coal_south_african` ($/mt) · `natural_gas_us` `natural_gas_europe` `liquefied_natural_gas_japan` ($/mmbtu) · `natural_gas_index` (2010=100)
음료	`cocoa` `coffee_arabica` `coffee_robusta` `tea_avg_3_auctions` `tea_colombo` `tea_kolkata` `tea_mombasa` ($/kg)
유지·박류	`coconut_oil` `groundnuts` `fish_meal` `groundnut_oil` `palm_oil` `palm_kernel_oil` `soybeans` `soybean_oil` `soybean_meal` `rapeseed_oil` `sunflower_oil` ($/mt)

분류(Pink Sheet 지수 그룹)	컬럼(단위)
곡물	`barley` `maize` `sorghum` `rice_thai_5` `rice_thai_25` `rice_thai_a_1` `rice_viet_namese_5` `wheat_us_srw` `wheat_us_hrw` ($/mt)
기타 식품	`banana_europe` `banana_us` `orange` `beef` `chicken` `lamb` `shrimps_mexican` `sugar_eu` `sugar_us` `sugar_world` ($/kg)
목재	`logs_cameroon` `logs_malaysian` `sawnwood_cameroon` `sawnwood_malaysian` ($/cubic meter) · `plywood` (cents/sheet)
기타 원자재	`cotton_a_index` `rubber_tsr20` `rubber_rss3` ($/kg) · `tobacco_us_import_u_v` ($/mt)
비료	`phosphate_rock` `dap` `tsp` `urea` `potassium_chloride` ($/mt)
금속·광물	`aluminum` `copper` `lead` `tin` `nickel` `zinc` ($/mt) · `iron_ore_cfr_spot` (헤더 단위 $/dmtu — **아래 주의**)
귀금속	`gold` `platinum` `silver` ($/troy oz)

- `iron_ore_cfr_spot` **단위·정의 주의**: 헤더 단위는 `$/dmtu` (1% Fe 단위)지만 xlsx Description 시트는 spot을 **건조 톤당 US$**로 정의한다 — 2008-12 이후 spot(62% Fe fines, cfr China), 2006~2008-11 spot(63.5% Fe), **그 이전은 연간 계약가(fob 브라질)가 월별로 반복**된 값. 실측 값도 톤당 수준이다 (2026-07 98.2, 최고 2021-06 214.4). 한 컬럼에 정의가 섞여 있으므로 추이 시연·임계값(§5-C `>= 150`은 38개월, 첫 달 2007-10)은 **2008-12 이후 구간**에서 해석한다.
- `coal_australian` : 호주 뉴캐슬 **일반탄(thermal, 6000 kcal/kg)** — 제철용 코크스탄이 아니다. 2022-02부터는 선물 가격, 그 전은 현물.
- **1960년부터 값이 없는 시리즈**(첫 유효 월): `crude_oil_wti` 1982-01 · `coal_australian` 1970-01 · `coal_south_african` 1984-01 · `liquefied_natural_gas_japan` · `natural_gas_index` 1977-01 · `groundnuts` 1980-01 · `fish_meal` · `wheat_us_srw` · `plywood` 1979-01 · `palm_kernel_oil` 1996-01 · `rapeseed_oil` · `sunflower_oil` 2002-02 · `rice_thai_25` · `rice_thai_a_1` 1986-01 · `rice_viet_namese_5` 2003-12 · `banana_europe` 1997-01 · `lamb` 1971-01 · `logs_cameroon` 1970-01 · `sawnwood_cameroon` 1970년 12개월만 있고 1971~1994 결측, 1995-01부터 연속 · `rubber_tsr20` 1999-01 · `dap` 1967-01. 나머지는 1960-01부터.
- **끝단·중간 결측**: `shrimps_mexican`은 **2023-09까지만** 있고 이후 33개월 NULL(시리즈 종료) · `tobacco_us_import_u_v` 2026-06·07 NULL · `sunflower_oil` 2002년 5개월, `rice_viet_namese_5` 2005~2009 11개월, `rice_thai_25` 4개월(#VALUE!) 결측.
- **정의가 바뀐 시리즈**(원본 헤더의 `**` 각주 — ETL이 이름에서 뗐다): `coal_south_african`, `groundnut_oil`, `beef`, `chicken`, `lamb`, `rubber_tsr20`, `potassium_chloride`, `fish_meal` (2026-07 대체 시리즈). 장기 추이를 해석할 때는 xlsx Description 시트를 본다.
- 철강 원가 시나리오에 바로 쓰이는 컬럼: `iron_ore_cfr_spot`, `coal_australian`, `coal_south_african`, `crude_oil_average`, `natural_gas_*`.

3-3. `commodity_trading.indices_monthly` — 799행 × 17열 (World Bank 지수, 2010=100)

`period_date` + 지수 16개. 원본 시트의 들여쓰기가 나타내는 계층은 다음과 같고, **컬럼은 전부 독립된 leaf 값**이다(상위 지수도 자기 값을 가진다).

```
total_index (= energy 67% + non_energy 33%, Index Weights 시트)
├─ energy
└─ non_energy
   ├─ agriculture
   │  ├─ beverages
   │  ├─ food ── oils_meals · grains · other_food
   │  └─ raw_materials ── timber · other_raw_mat
   ├─ fertilizers
   └─ metals_minerals ── base_metals_ex_iron_ore
precious_metals   (별도 지수 — 총지수 가중치 배분에 들어가지 않는다: Index Weights 시트 기준)
```

전부 1960-01부터 값이 있다(결측 0).

3-4. `commodity_trading.prices_legacy` — 449행 × 64열 (IMF 계열, 2017-06까지)

`date`(`'YYYY-MM-01'`) + **지수 10개** + **상품 53개**. 지수는 2005=100(2005년 평균이 정확히 100.0 — 실측). 단위는 파일에 없다. **Pink Sheet와 다른 계열**이므로 같은 이름의 상품이라도 값을 이어 붙이지 말고, 쓰려면 별도 개념으로 둔다. 결측은 NULL(원본의 문자열 `nan` 1,143셀 = 지수 679 + 상품 464).

- 지수(10): `all_commodity_price_index` `non_fuel_price_index` `food_and_beverage_price_index` `food_price_index` `beverage_price_index` `industrial_inputs_price_index` `agricultural_raw_materials_index` `metals_price_index` `fuel_energy_index` **`crude_oil_petroleum`** — 마지막 것은 이름과 달리 **원유 가격지수(2005=100)**이지 가격이 아니다(2005년 평균 100.0). 상품으로 매핑하지 말 것. 시작월: `all_commodity` · `fuel_energy` · `crude_oil_petroleum` 계열은 1992-01, `non_fuel` · `food_and_beverage` · `food`는 1991-01, 나머지는 1980-02.
- 상품(53): `aluminum` `bananas` `barley` `beef` `coal` `cocoa_beans` `coffee_other_mild_arabicas` `coffee_robusta` `rapeseed_oil` `copper` `cotton` `fishmeal` `groundnuts_peanuts` `hides` `china_import_iron_ore_fines_62_fe_spot` `lamb` `lead` `soft_logs` `hard_logs` `maize_corn` `natural_gas_russian_natural_gas_border_price_in_germany` (1985-01~) `natural_gas_indonesian_liquefied_natural_gas_in_japan` (1992-01~) `natural_gas_spot_price_at_the_henry_hub_terminal_in_louisiana` (1991-01~) `nickel` `crude_oil_petroleum_simple_average_of_three_spot_prices` `crude_oil_petroleum_dated_brent_light_blend` `oil_dubai` `crude_oil_petroleum_west_texas_intermediate_40_api` `olive_oil` `oranges` `palm_oil` `swine_pork` `poultry_chicken` `rice` `rubber` `fish_salmon` `hard_sawnwood` `soft_sawnwood` `shrimp` `soybean_meal` `soybean_oil` `soybeans` `sugar_european_import_price` (1991-01~) `sugar_free_market` `sugar_u_s_import_price` `sunflower_oil` `tea` `tin` `uranium` `wheat` `wool_coarse` `wool_fine` `zinc`

4. 타입·형식 규칙 (매핑할 때 매번 적용)

상황	규칙
모든 컬럼이 TEXT	매핑 SELECT에서 `CAST(`col`AS double)` / `int` / `date`. 통신·금융 사례 SQL과 같은 방식. `double`·`date`·`int` 세 가지는 9/3 이 데이터소스에서 실행해 통했다(부록 B). Timbr 공식 문서는 타입을 열거하지 않으므로 다른 타입이 필요하면 블록 하나로 먼저 시험
날짜	`period_date`, `date`는 `'1960-01-01'` 형식 문자열 → `CAST(`period_date`AS date)`. 월 단위라 항상 1일. 조건절은 `>= '2026-06-01'`(문자열)도, `>= CAST('2020-01-01' AS date)`도 통했다

상황	규칙
0/1 플래그	`CAST(` Machine failure `AS int)` 또는 boolean이 필요하면 `(` Machine failure `= '1')`
결측	NULL 그대로 둔다. 빈 문자열 셀은 없으므로 `nullif(` col `, '')` 는 필요 없다
컬럼명에 공백·대괄호	Timbr SQL은 백틱 `` `Air temperature [K]` ``, 원본 스키마 조회(SQL Lab 드롭다운 = 원본 스키마)는 큰따옴표 `"Air temperature [K]"`
문자열 리터럴	반드시 작은따옴표 `'M'`. 백틱은 식별자 전용
개념 속성명	백틱 안에 공백이 든 원본명을 그대로 써도 되지만, 질의 편의상 `air_temp_k` 처럼 별칭을 권장(매핑 SELECT의 `AS`)

5. 매핑 절차 — 두 가지 길

A. 콘솔 마법사(가장 빠름, SQL 없이)

1. 온톨로지 선택(`3_manufacture` 또는 `4_commodity_trading`) — 데이터소스는 이미 붙어 있으니 건드리지 않는다.
2. `Create → Concepts from datasources`(공식 문서 표기: "Create concepts in bulk from datasource") → 스키마 `manufacture`(또는 `commodity_trading`) → 테이블 선택.
3. **Primary Key**: 제조는 `UDI`, 상품거래 3개 테이블은 `period_date`(레거시는 `date`).
4. **Entity Label**: 제조는 `Product ID`, 상품거래는 `period_date`(월별 시계열이라 PK 외에 읽을 이름이 없다 — 보통은 PK 코드값을 Label로 쓰지 말라고 안내하지만 여기선 대안이 없다).
5. 마법사는 속성을 **전부 STRING으로** 만든다(기존 사례 실측). 생성 후 속성 편집에서 숫자/날짜로 바꾸거나, §5-B처럼 SQL 매핑에서 CAST한다.
6. 생성 후 SQL Lab에서 SCHEMA 드롭다운을 **`dtimbr`**로 두고 `SELECT count(*) FROM` dtimbr`.`개념명`` — 원본 행수(10,000 / 799 / 799 / 449)와 같아야 정상.
7. 컬럼 일부를 떼어 참조 개념을 만들 때(`Map Data → Create Concept Mapping`)는 **`Map distinct values` 토글 ON**. `Concepts from property`는 속성의 고유값마다 개념을 하나씩 만드는 기능이니 `Type`(L/M/H)에는 쓰지 않는다(§5-B처럼 참조 개념 하나로).

B. SQL Lab — 제조 (9/3 실측 통과)

아래 블록은 통신·금융 온톨로지의 형식을 따랐고, **9/3에 임시 온톨로지(같은 `bootcamp_supabase`)에서 REST로 전부 실행해 주석의 행수를 확인했다**(부록 B). SQL Lab에 붙일 때는 블록 단위로, 세미콜론은 빼도 된다.

```
-- 1) 품질등급 참조 개념 (L/M/H 3행) — 관계를 보여주기 위한 작은 개념
CREATE OR REPLACE CONCEPT `quality_grade` (
    `grade` string,
    PRIMARY KEY(`grade`), LABEL(`grade`)
) INHERITS (`thing`) DESCRIPTION '제품 품질 등급 L(low)/M(medium)/H(high)';

CREATE OR REPLACE MAPPING `map_quality_grade` INTO `quality_grade` AS
    SELECT DISTINCT `Type` AS `grade`
```

```
    FROM `manufacture`.`ai4i_predictive_maintenance`;               -- 3행

-- 2) 공정 사이클 개념 (10,000행) + 등급 관계
CREATE OR REPLACE CONCEPT `process_cycle` (
    `udi` int, `product_id` string, `grade` string,
    `air_temp_k` double, `process_temp_k` double, `rotational_speed_rpm` int,
    `torque_nm` double, `tool_wear_min` int,
    `machine_failure` int, `twf` int, `hdf` int, `pwf` int, `osf` int, `rnf` int,
    PRIMARY KEY(`udi`), LABEL(`product_id`),
    CONSTRAINT `of_grade` FOREIGN KEY (`grade`) REFERENCES `quality_grade`(`grade`)
        INVERSEOF `cycles`
) INHERITS (`thing`) DESCRIPTION '공정 사이클 1건(AI4I 2020, 합성). 온도·회전속도·토크·공구마모와 고장 플

CREATE OR REPLACE MAPPING `map_process_cycle` INTO `process_cycle` AS
    SELECT CAST(`UDI` AS int)                    AS `udi`,
           `Product ID`                          AS `product_id`,
           `Type`                                AS `grade`,
           CAST(`Air temperature [K]` AS double)     AS `air_temp_k`,
           CAST(`Process temperature [K]` AS double) AS `process_temp_k`,
           CAST(`Rotational speed [rpm]` AS int)     AS `rotational_speed_rpm`,
           CAST(`Torque [Nm]` AS double)             AS `torque_nm`,
           CAST(`Tool wear [min]` AS int)            AS `tool_wear_min`,
           CAST(`Machine failure` AS int) AS `machine_failure`,
           CAST(`TWF` AS int) AS `twf`, CAST(`HDF` AS int) AS `hdf`, CAST(`PWF` AS int) AS `pwf`,
           CAST(`OSF` AS int) AS `osf`, CAST(`RNF` AS int) AS `rnf`
    FROM `manufacture`.`ai4i_predictive_maintenance`;              -- 10,000행

-- 3) 하위개념 = 조건 필터 뷰 (복제 없음)
CREATE OR REPLACE CONCEPT `failed_cycle`
    INHERITS (`process_cycle`) FROM `timbr`.`process_cycle`
    WHERE `machine_failure` = 1;                                    -- 339행

CREATE OR REPLACE CONCEPT `heat_dissipation_failure`
    INHERITS (`failed_cycle`) FROM `timbr`.`failed_cycle`
    WHERE `hdf` = 1;                                                -- 115행 (2단 상속)

-- RNF는 라벨 0인 행이 18/19라 failed_cycle 아래에 두면 1행뿐 — process_cycle에서 직접 파생 (미실행)
-- CREATE OR REPLACE CONCEPT `random_failure`
--     INHERITS (`process_cycle`) FROM `timbr`.`process_cycle` WHERE `rnf` = 1;   -- 기대 19행

-- 4) 측정값 (재실행 시 이미 있으면 CREATE OR REPLACE MEASURE — 기존 사례 실측 구문)
CREATE MEASURE `cycle_count` bigint AS SELECT count(1);
CREATE MEASURE `failure_count` bigint AS SELECT sum(`machine_failure`);
```

검증·시연 질의(SCHEMA = `dtimbr`, 관계 경로는 **하나의 백틱 쌍**으로) — 결과는 9/3 실측값:

```
SELECT count(*) FROM `dtimbr`.`process_cycle`;          -- 10000
SELECT count(*) FROM `dtimbr`.`failed_cycle`;           -- 339
SELECT `grade`, AGGREGATE(`measure.cycle_count`) AS cycles, AGGREGATE(`measure.failure_count`) AS failur
FROM `dtimbr`.`process_cycle` GROUP BY `grade`;          -- H 1003/21 · M 2997/83 · L 6000/235
SELECT `of_grade[quality_grade].grade` AS g, avg(`torque_nm`) AS avg_torque
FROM `dtimbr`.`failed_cycle` GROUP BY `of_grade[quality_grade].grade`;   -- H 50.9 · L 50.8 · M 48.2 (GR
SELECT `grade`, count(`cycles[process_cycle].udi`) AS n
FROM `dtimbr`.`quality_grade` GROUP BY `grade`;          -- 역방향 관계 경로: H 1003 · M 2997 · L 6000
```

선택 블록 — **규칙으로 플래그를 재현하는 하위개념**(§3-1 표). Postgres에서 계산한 행수는 플래그와 정확히 같았고, Timbr 하위개념으로는 아직 실행하지 않았다(WHERE에 산술식이 들어가는 형태 — 안 되면 매핑 SELECT에서 파생 컬럼으로 만든다):

```
CREATE OR REPLACE CONCEPT `hdf_by_rule`
    INHERITS (`process_cycle`) FROM `timbr`.`process_cycle`
    WHERE (`process_temp_k` - `air_temp_k`) < 8.6 AND `rotational_speed_rpm` < 1380;   -- 기대 115행 = H
```

C. SQL Lab — 상품거래 (9/3 실측 통과, 넓은 표를 그대로)

월 1행 구조를 그대로 개념으로 올리는 가장 단순한 형태. 컬럼 71개를 전부 나열해도 되지만 시나리오에 필요한 것만 골라도 된다(아래는 철강 원가 시나리오용 발췌). **참조되는 개념(`price_index_month`)을 먼저 만든다.**

```
-- 1) 지수 개념 (월 키) — 아래 commodity_month가 관계로 참조하므로 먼저 실행
CREATE OR REPLACE CONCEPT `price_index_month` (
    `period_date` date, `total_index` double, `energy` double, `non_energy` double,
    `agriculture` double, `fertilizers` double, `metals_minerals` double,
    `base_metals_ex_iron_ore` double, `precious_metals` double,
    PRIMARY KEY(`period_date`), LABEL(`period_date`)
) INHERITS (`thing`) DESCRIPTION 'World Bank 상품가격 지수(2010=100), 월별';

CREATE OR REPLACE MAPPING `map_price_index_month` INTO `price_index_month` AS
    SELECT CAST(`period_date` AS date) AS `period_date`,
           CAST(`total_index` AS double) AS `total_index`, CAST(`energy` AS double) AS `energy`,
           CAST(`non_energy` AS double) AS `non_energy`, CAST(`agriculture` AS double) AS `agriculture`,
           CAST(`fertilizers` AS double) AS `fertilizers`, CAST(`metals_minerals` AS double) AS `metals_
           CAST(`base_metals_ex_iron_ore` AS double) AS `base_metals_ex_iron_ore`,
           CAST(`precious_metals` AS double) AS `precious_metals`
    FROM `commodity_trading`.`indices_monthly`;                      -- 799행

-- 2) 월별 가격 개념 (철강 원가 시나리오용 발췌) + 같은 월 지수로의 관계(월 키 1:1)
CREATE OR REPLACE CONCEPT `commodity_month` (
    `period_date` date, `iron_ore_usd_dmt` double, `coal_australian_usd_mt` double,
    `coal_south_african_usd_mt` double, `crude_oil_avg_usd_bbl` double,
    `natural_gas_us_usd_mmbtu` double, `copper_usd_mt` double, `aluminum_usd_mt` double,
    `nickel_usd_mt` double, `gold_usd_troy_oz` double,
    PRIMARY KEY(`period_date`), LABEL(`period_date`),
    CONSTRAINT `index_of_month` FOREIGN KEY (`period_date`) REFERENCES `price_index_month`(`period_date`
        INVERSEOF `prices_of_month`
) INHERITS (`thing`) DESCRIPTION 'World Bank Pink Sheet 월별 원자재 가격(명목 USD). 단위는 속성명에 표기

CREATE OR REPLACE MAPPING `map_commodity_month` INTO `commodity_month` AS
    SELECT CAST(`period_date` AS date)            AS `period_date`,
           CAST(`iron_ore_cfr_spot` AS double)    AS `iron_ore_usd_dmt`,
           CAST(`coal_australian` AS double)      AS `coal_australian_usd_mt`,
           CAST(`coal_south_african` AS double)   AS `coal_south_african_usd_mt`,
           CAST(`crude_oil_average` AS double)    AS `crude_oil_avg_usd_bbl`,
           CAST(`natural_gas_us` AS double)       AS `natural_gas_us_usd_mmbtu`,
           CAST(`copper` AS double)               AS `copper_usd_mt`,
           CAST(`aluminum` AS double)             AS `aluminum_usd_mt`,
           CAST(`nickel` AS double)               AS `nickel_usd_mt`,
           CAST(`gold` AS double)                 AS `gold_usd_troy_oz`
    FROM `commodity_trading`.`prices_monthly`;                       -- 799행

-- 3) 하위개념 예: 철광석 고가 국면 (38개월, 첫 달 2007-10 — 2008-12 이전 값은 정의가 달라 주의)
```

```
CREATE OR REPLACE CONCEPT `high_iron_ore_month`
    INHERITS (`commodity_month`) FROM `timbr`.`commodity_month`
    WHERE `iron_ore_usd_dmt` >= 150;
```

```
-- 검증/시연 질의 (dtimbr) — 9/3 실측: 2026-07 98.2 / 131.9 / metals_index 140.5
SELECT `period_date`, `iron_ore_usd_dmt`, `coal_australian_usd_mt`,
       `index_of_month[price_index_month].metals_minerals` AS metals_index
FROM `dtimbr`.`commodity_month`
WHERE `period_date` >= CAST('2020-01-01' AS date) ORDER BY `period_date` DESC LIMIT 24;
```

D. (제안, 미구축) 긴 형식(long) 뷰 — 시계열 질의를 쉽게

"철광석 가격 추이", "구리와 알루미늄 상관"처럼 **상품을 행으로 다루는 질의**는 넓은 표(71열)보다 `(period_date, commodity, category, unit, price)` 형태가 낫다. Timbr 매핑 SELECT에는 서브쿼리를 넣을 수 없었으므로(MySQL 온라인 데모에서 반증실험으로 확인 — 구조적 거부, 공식 문서에는 언급 없음) **Postgres 쪽에 뷰**를 만들어 매핑하는 것이 정석이다: `commodity_trading.v_prices_long`, `v_indices_long`. 그러면 `commodity`(상품 마스터: 이름·분류·단위)와 `price_observation`(월×상품, FK→commodity) 두 개념과 관계로 자연스러운 그래프가 된다. 필요하면 박종범이 뷰를 만들어 이 문서에 추가한다.

6. 검증 습관

- 개념 생성 직후 `SELECT count(*) FROM dtimbr.개념` — 원본 행수와 대조. 안 맞으면 매핑의 Map `distinct values`·WHERE부터 본다.
- 숫자 집계가 `column does not exist`로 실패하면 SCHEMA 드롭다운이 `dtimbr`인지, measure에 `AGGREGATE()`를 씌웠는지 확인.
- 관계 경로는 `` `rel[target].prop` `` 처럼 **경로 전체를 하나의 백틱 쌍**으로(공식 FAQ: 경로 전체를 인용 부호로 감싼다). 세그먼트별로 감싸면 파서가 관계로 읽지 못한다(7/23 실증).
- 관계 방향: §5-B 같은 **개념 정의 안의 단일 FK**는 정의한 방향대로 동작했다(`of_grade[…]`와 역방향 `cycles[…]` 둘 다 9/3 실측). **다대다(MULTIREFERENCE) 관계 매핑**에서는 FK 제약 이름과 방향이 반대로 읽힌 실측이 있으니(통신 사례 8/24) 그 형태를 쓸 때만 주의.

7. DB 직접 접속(선택) — 매핑에는 필요 없다

Timbr 안에서 원본 조회까지 되므로 매핑 작업에는 아래가 필요 없다. DBeaver·psql로 직접 보고 싶을 때만.

항목	값
종류	PostgreSQL 17 (Supabase)
호스트	`aws-0-ap-southeast-1.pooler.supabase.com` (**Session Pooler** — 직접연결 `db.<ref>.supabase.co`는 IPv6 전용이라 일반 환경에서 안 붙는다)
포트	5432
데이터베이스	`postgres`

항목	값
사용자·비밀번호	**이 문서에 싣지 않는다.** 필요한 분은 박종범에게 요청 — 읽기전용 계정을 따로 만들어 전달할 수 있다
SSL	`sslmode=require`
스키마	`manufacture`, `commodity_trading` (그 외 스키마는 다른 사례용 — 건드리지 말 것)

- 읽기 전용으로 쓰기 바란다. 테이블을 지우거나 바꾸면 Timbr 매핑이 깨진다. 재적재 스크립트는 코세나 저장소(박종범 관리)에 있다.
- 무료 티어라 일정 기간 미사용 시 **일시정지**될 수 있다(통신·금융 사례에서 실제로 겪음). 붙지 않으면 박종범에게 연락.

8. 하지 말 것 · 함정

- **데이터소스를 새로 만들지 않는다.** `bootcamp_supabase`가 두 온톨로지에 이미 연결·활성 상태다. 새 온톨로지를 추가로 만들면 `ALTER ONTOLOGY` 이름 `ADD DATASOURCE` bootcamp_supabase `` `로 같은 것을 붙이고, 새로 붙인 데이터소스는` is_active `가 0으로 들어오므로(9/3 실측) 콘솔 위저드 기본값을 위해` SET active_datasource = 'bootcamp_supabase'`를 실행한다.
- `CREATE DATASOURCE`를 굳이 다시 쓴다면 `TYPE 'postgresql'`이 이 인스턴스에서 통했다(`'postgres'`는 에러; 공식 문서는 TYPE 값을 열거하지 않는다). `WITH VIRTUALIZATION`은 Spark/Databricks 또는 Timbr Virtualization Service가 있을 때만 붙인다.
- 제조 데이터를 발표할 때 **"합성 데이터"**를 밝힌다. 고장률 3.4% 같은 수치는 이 데이터셋 안의 값이지 어떤 공장의 실측이 아니다. UCI 설명문의 등급 비율·TWF/RNF 건수는 파일과 다르니 원문 수치를 인용하지 않는다.
- 레거시(IMF)와 Pink Sheet(World Bank)를 **하나의 시계열로 잇지 않는다.** 같은 상품명이라도 정의가 다르다. 레거시는 내부 실습 한정(§2 라이선스).
- 철광석 컬럼은 2008-12 전후로 정의가 다르다(§3-2). `crude_oil_petroleum`(레거시)은 지수다(§3-4).
- Pink Sheet는 매월 갱신된다. 우리 적재본은 2026-08-04 갱신본(2026-07까지)이며, 최신이 필요하면 재적재(박종범).
- `Concepts from property` 마법사는 속성 고유값마다 개념을 만든다 — `Type`(L/M/H)에 쓰지 말고 §5-B처럼 참조 개념으로.

9. 변경 이력

날짜	내용
2026-09-02	데이터 적재(Supabase `manufacture`, `commodity_trading`), `3_manufacture`에 데이터소스 연결
2026-09-03	`4_commodity_trading` 생성, 공용 데이터소스 `bootcamp_supabase`로 통일, 이 문서 작성. 적대적 검증 2축(내부 정합·외부 인용) + 임시 온톨로지 실행 검증 반영(부록 B)

부록 A. 이 문서의 수치를 낸 조회 (Supabase 원본 스키마, 9/3)

```
-- 제조: 분포·고장·범위
SELECT "Type", count(*) FROM manufacture.ai4i_predictive_maintenance GROUP BY 1;
SELECT sum(("Machine failure"='1')::int), sum(("TWF"='1')::int), sum(("HDF"='1')::int),
       sum(("PWF"='1')::int), sum(("OSF"='1')::int), sum(("RNF"='1')::int)
FROM manufacture.ai4i_predictive_maintenance;                        -- 339, 46, 115, 95, 98, 19
SELECT count(distinct "UDI"), count(distinct "Product ID") FROM manufacture.ai4i_predictive_maintenance;
-- 라벨 vs 플래그
SELECT sum((("TWF"::int+"HDF"::int+"PWF"::int+"OSF"::int+"RNF"::int) >= 1))::int) AS any_flag,     -- 3
       sum(("Machine failure"::int = 1 AND ("TWF"::int+"HDF"::int+"PWF"::int+"OSF"::int+"RNF"::int) = 0)
       sum(("RNF"::int = 1 AND "Machine failure"::int = 0)::int) AS rnf_label0                        --
FROM manufacture.ai4i_predictive_maintenance;
-- 규칙 재현(HDF)
SELECT count(*) FROM manufacture.ai4i_predictive_maintenance
WHERE ("Process temperature [K]"::float - "Air temperature [K]"::float) < 8.6
  AND "Rotational speed [rpm]"::int < 1380;                          -- 115 (HDF 플래그와 동일 행)
-- 상품거래: 시작월·결측·최고점
SELECT min("period_date") FILTER (WHERE "crude_oil_wti" IS NOT NULL) FROM commodity_trading.prices_month
SELECT "period_date", "iron_ore_cfr_spot" FROM commodity_trading.prices_monthly
ORDER BY "iron_ore_cfr_spot"::float DESC NULLS LAST LIMIT 1;         -- 2021-06-01, 214.4
SELECT count(*), min("period_date") FROM commodity_trading.prices_monthly WHERE "iron_ore_cfr_spot"::flo
SELECT avg("all_commodity_price_index"::float) FROM commodity_trading.prices_legacy WHERE "date" LIKE '2
```

부록 B. 검증 이력 (2026-09-03)

- **실행 검증**: 임시 온톨로지 `zz_probe_mapping_0903`을 `CREATE ONTOLOGY`로 만들어 `bootcamp_supabase`를 붙이고, §5-B·§5-C의 모든 CREATE CONCEPT / MAPPING / 하위개념 / MEASURE와 검증 질의를 REST(`/timbr/openapi/ontology/<name>/query`)로 실행 — 전부 성공, 행수는 본문 주석과 같음(10,000 / 3 / 339 / 115 / 799 / 799 / 38). 실행 후 `DROP ONTOLOGY`로 삭제(온톨로지 목록 10개로 복귀 확인). SQL Lab UI에서는 미실행.
- **내부 정합 검증**(저장소 1차 자료·검증된 선례 대조): 레거시 지수/상품 분류(9+54 → 10+53, `crude_oil_petroleum`은 지수), `Machine failure`≠플래그 OR, SQL Lab 드롭다운 우선, CC BY 3자 단서, 담배 분류, 철광석 단위, 시리즈 끝단 결측, 고객사명 제거, TYPE 표현 등 정정.
- **외부 인용 검증**(UCI·World Bank·Timbr 공식 문서·GitHub 리포지토리 재조회): UCI 등급 비율·고장 규칙·인용 형식, Pink Sheet 철광석 정의·석탄 정의·귀금속 가중치·URL 갱신 방식·표기 형식, 레거시 리포지토리 출처·라이선스 상충·기준연도, Timbr 타입 미열거·`SET active_datasource` 영속 미문서·마법사 공식 명칭 — 정정 반영.
- **확인하지 못한 것**: World Bank 3자 시계열의 재배포 조건 · `SET active_datasource`의 영속 여부(문서 부재) · 콘솔 마법사에서 속성 타입을 바꾸는 UI의 실측 · 규칙 기반 하위개념(`hdf_by_rule`)의 Timbr 실행 · 레거시 CSV 원본 헤더의 "Price index" 표기 여부.

코세나 내부·동료 공유용. 데이터는 전부 공개 데이터(UCI 합성·World Bank·IMF 계열)이며 고객 데이터를 포함하지 않는다.